Übungsblatt 03: POP3 und HTTP mit Python Sockets

PR Rechnernetze und Netzwerkprogrammierung 621.801/621.802

Raphael Bleier, Devis, Fabio, Gruppe 1, WS 2026/2027

Aufgabe 3.1

POP3 anhand RFC 1939 untersuchen: Arbeitsablauf, Sicherheit und Unterschied zwischen single-line und multi-line response. Mit IMAP vergleichen.

Nach dem TCP-Verbindungsaufbau schickt der Server eine Begrüßung mit `+OK`. Es folgen drei Zustände:

1. AUTHORIZATION: Der Client sendet die POP3-Kommandos `USER` und `PASS`; der Server prüft die Anmeldung und sperrt den Maildrop exklusiv. Erst bei Erfolg beginnt TRANSACTION (RFC 1939, Abschnitt 4).
2. TRANSACTION: `STAT` nennt Anzahl und Gesamtgröße; `LIST` listet Größen und `RETR n` liefert Nachricht n. `DELE n` markiert eine Nachricht zunächst nur. `RSET` entfernt diese Markierungen (Abschnitt 5).
3. UPDATE: `QUIT` aus TRANSACTION übernimmt die Löschungen und beendet die Verbindung. Bei einem Abbruch ohne dieses `QUIT` darf der Server die Markierungen nicht übernehmen (Abschnitt 6). `QUIT` vor der Anmeldung beendet die Verbindung ohne UPDATE.

Erklärung

Der Maildrop ist hier der Vorrat an Nachrichten eines Benutzers. Die Sperre verhindert, dass zwei Sitzungen gleichzeitig verschiedene Löschstände auf denselben Vorrat anwenden. Abrufen heißt noch nicht Löschen: Das sind getrennte Kommandos.



Unverschlüsseltes POP3 (üblicherweise auf Port 110) schützt weder Passwort noch Nachrichten. Ein Mitschnitt kann beides offenlegen. Für echte Zugänge ist TLS mit Zertifikatsprüfung nötig, etwa implizites TLS auf Port 995. APOP aus RFC 1939 ist keine Verschlüsselung der Maildaten und kein Ersatz für TLS. Der hier implementierte Server ist auf einen lokalen Labortest ohne TLS beschränkt.

Erklärung

Authentifizierung und Verschlüsselung lösen verschiedene Probleme. Ein richtig geprüftes Passwort verhindert nicht, dass jemand den unverschlüsselten Transport mitliest. Deshalb werden im Versuch ausschließlich die vorgegebenen Testdaten verwendet.

Eine single-line response enthält genau eine Statuszeile mit `+OK` oder `-ERR`. Eine erfolgreiche multi-line response beginnt ebenfalls mit einer Statuszeile, enthält anschließend Datenzeilen und endet mit einer Zeile, die nur einen Punkt enthält. Jede Zeile endet mit CRLF. Datenzeilen mit führendem Punkt erhalten einen zusätzlichen Punkt; der Client entfernt beim Lesen genau diesen einen Punkt. `LIST` ist ohne Argument mehrzeilig, `LIST 1` einzeilig. Eine negative Antwort hat keinen anschließenden Datenblock.

Erklärung

Die Punktzeile ist ein Endezeichen, keine Nachrichtendatei. Eine echte Mailzeile mit einem einzelnen Punkt wird daher als zwei Punkte übertragen. Auch Leerzeilen am Nachrichtenende gehören zum Inhalt und dürfen nicht durch pauschales Abschneiden verloren gehen.

IMAP verwendet getaggte Kommandos und Abschlüsse sowie ungetaggte Antworten. Literale können eine explizite Bytelänge haben; sie werden nicht durch POP3-Dot-Stuffing eingerahmt. IMAP unterstützt serverseitige Ordner, Flags und Synchronisierung mehrerer Geräte. POP3 konzentriert sich auf den Abruf. Serverkopien können dabei erhalten bleiben, wenn der Client keine Löschung verlangt. Grundlage: RFC 9051 sowie LV-Folien 36–37.

Erklärung

Ein Tag wie `A001` ordnet bei IMAP das Ergebnis einer Anfrage zu. Das ist etwas anderes als die Nachrichtennummer bei POP3. Für den Vergleich zählt vor allem, wo Ordner und Lesestatus verwaltet werden.

Aufgabe 3.2

Einen POP3-Client mit Standardbibliothek und Socket-Kommunikation schreiben. Anmelden, alle Nachrichten auflisten, eine Nachricht abrufen und die Verbindung ordentlich abbauen. Mit einem geeigneten Server testen; lokale Server oder ein TLS-Tunnel sind erlaubt.

Abgabe: `pop3_client.py`. Der Ablauf lautet Begrüßung, `USER`, `PASS`, `LIST`, `RETR n`, `QUIT`. Statusfehler werden abgefangen, CRLF und Dot-Stuffing ausgewertet. Das Passwort wird verdeckt abgefragt. Der Client schließt Reader und Socket auch nach einer fehlerhaften Begrüßung oder einem fehlgeschlagenen `QUIT`.

Erklärung

TCP liefert einen Bytestrom, keine vollständigen Antworten pro Empfangsaufruf. Der gepufferte Reader setzt deshalb auch fragmentierte Zeilen wieder zusammen. Die Statusprüfung kommt vor dem Lesen eines Datenblocks, weil eine Fehlermeldung gerade keinen mehrzeiligen Block ankündigt.

Der Test gegen den lokalen Server mit der Original-Datenbank bestätigte vier Nachrichten. `LIST` meldete 184, 185, 154 und 164 Bytes. Der erste `RETR`-Abruf stimmte bytegenau mit der CRLF-normalisierten Originalnachricht überein. Weitere Tests prüfen fragmentierte Antworten, Punktzeilen, EOF und die Abwehr von Zeilenumbrüchen in Kommandos. Eine Gegenprobe mit dem im Blatt verlinkten GoPop3Server bestätigte Anmeldung, `LIST`, `RETR` und `QUIT`. Dieser separate Referenzserver lieferte zwei Nachrichten mit 151 und 115 Bytes; der erste Betreff war „Hello Alice“. Der Versuch lief in einem isolierten lokalen Netzwerk.

Erklärung

Die Größen beziehen sich auf die Nachricht mit CRLF, nicht auf Statuszeilen, zusätzliche Punkte oder die abschließende Punktzeile. Der Vergleich mit den Originalbytes prüft daher eine andere Eigenschaft als ein bloßer Vergleich des sichtbaren Mailtexts.

Aufgabe 3.3

Einen minimalen POP3-Server mit der bereitgestellten Klasse SampleDataBase.py implementieren. Die Nachrichten sollen im Posteingang eines echten Mailprogramms erscheinen. Pro Benutzer einen Thread verwenden; parallele Benutzer zulassen. Client und Server praktisch testen und den Ablauf mit Screenshots dokumentieren. RFC 1939 und RFC 2449 beachten.

Abgabe: `pop3_server.py`. Die externe Datenbank wird über `authenticate` und `get_messages` benutzt. Pro Verbindung läuft ein Thread. Ein Lock pro Benutzer verhindert zwei gleichzeitige Transaktionen auf demselben Maildrop. Unterschiedliche Benutzer bleiben parallel bedienbar.

Erklärung

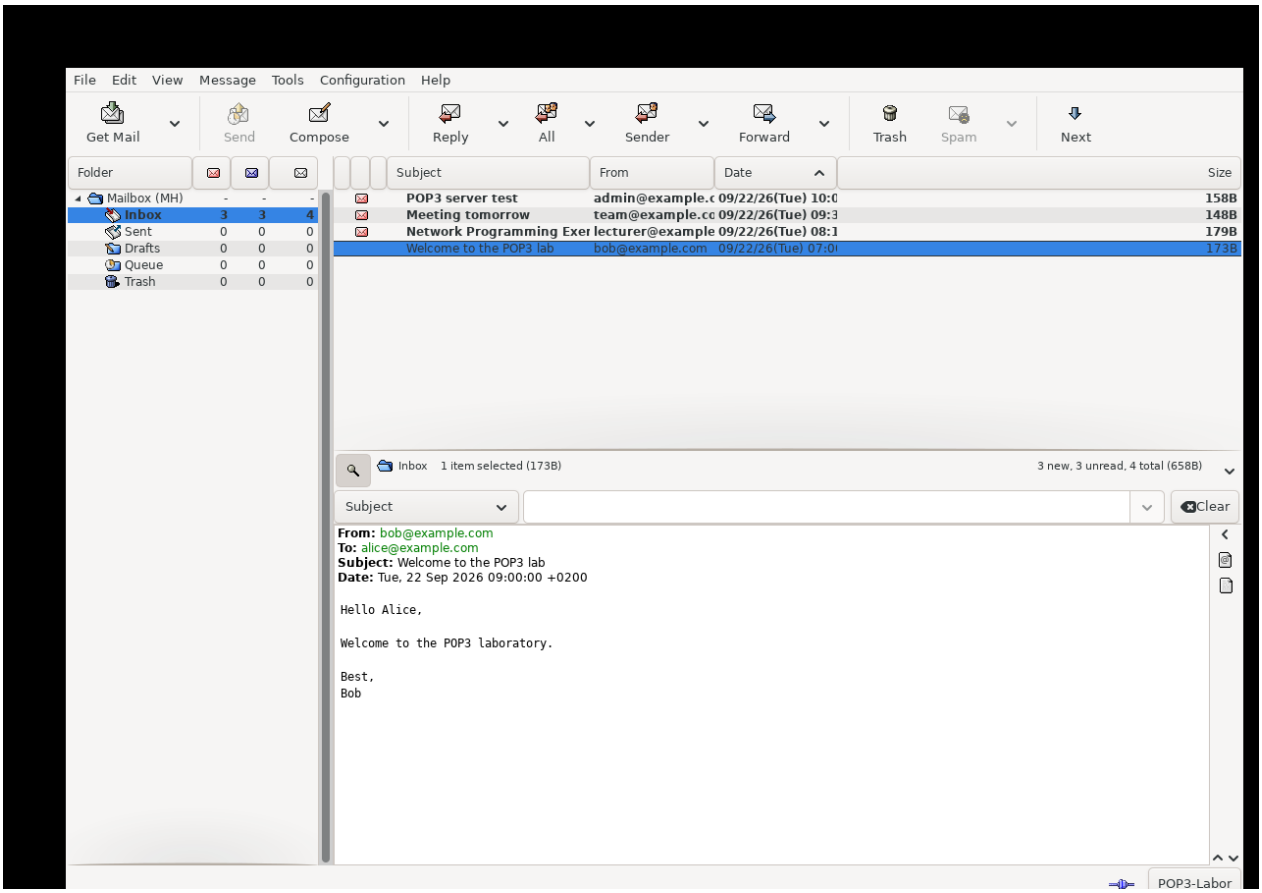
Die Threads gehören zu Verbindungen, nicht dauerhaft zu Konten. Wenn sich derselbe Benutzer später erneut verbindet, übernimmt ein neuer Sitzungsthread die Bearbeitung, während der gemeinsame Benutzer-Lock weiterhin verhindert, dass zwei gleichzeitig angemeldete Sitzungen widersprüchliche Löschungen am selben Maildrop vornehmen. Das Annehmen weiterer Verbindungen bleibt davon getrennt.

Implementiert sind `USER`, `PASS`, `STAT`, `LIST`, `RETR`, `DELE`, `RSET`, `NOOP` und `QUIT`; zusätzlich `CAPA`, `TOP` und `UIDL`. Nachrichtennummern bleiben innerhalb einer Sitzung stabil. `UIDL` nutzt einen Hash aus Original-ID und Inhalt. Löschungen werden nur bei `QUIT` übernommen. Die Datenbankdatei bleibt unverändert; ohne dauerhafte Löschschnittstelle besteht der Löschstand nur im laufenden Serverprozess. Das Inaktivitätstimeout beträgt 600 Sekunden.

Erklärung

`DELE` entfernt zunächst keine Datei. Die Sitzung merkt sich nur die Nummer. Erst beim Wechsel nach UPDATE wird die ursprüngliche Datenbank-ID in den lokalen Löschstand geschrieben. Dadurch kann ein abgebrochener Download keine bereits markierten Nachrichten versehentlich entfernen.

GUI-Test: Claws Mail 4.3.1 mit einem frischen Profil, POP3 auf Loopback, Port 11110, Testbenutzer aus der Original-Datenbank und ohne TLS. Die Option zum Löschen nach Abruf war ausgeschaltet. Nach dem Abruf erschienen vier Nachrichten im Posteingang. Die erste Nachricht wurde geöffnet. Der Screenshot zeigt Betreffe, Header und Text. Claws zeigt Größen von 158, 148, 179 und 173 B (658 B gesamt); diese GUI-Werte weichen von den durch `LIST` gelieferten Octet-Zahlen ab und gelten nicht als deren Nachweis. Die Standardbibliothek `poplib` wurde zusätzlich als unabhängiger Protokollclient verwendet.



Aufgabe 3.4

Einen Single-Thread-HTTP/0.9-Server schreiben. Port (Vorgabe 80) und document root als Kommandozeilenparameter anbieten. Einzelne Dateien liefern, für Verzeichnisse index.html verwenden und bei Fehlern ohne Rückmeldung schließen. Mit documentRoot.zip und unterschiedlichen Browsern testen. Werden die Daten richtig angezeigt?

Abgabe: `http_server.py -mode 09`. Dieser Modus behandelt Verbindungen nacheinander. Akzeptiert wird `GET /pfad` mit CRLF. Die Antwort besteht nur aus den Dateibytes; danach wird die Verbindung geschlossen. Pfade werden dekodiert, aufgelöst und auf das Basisverzeichnis beschränkt. Fehlende Dateien und ungültige Anfragen liefern keine Antwortbytes.

Erklärung

HTTP/0.9 kennt in dieser einfachen Form weder Statuszeile noch Header. Der Client erkennt das Ende deshalb am Verbindungsabbau. Ein HTTP/1.0-Statuscode als Fehlermeldung würde gerade nicht zum geforderten Verhalten passen.

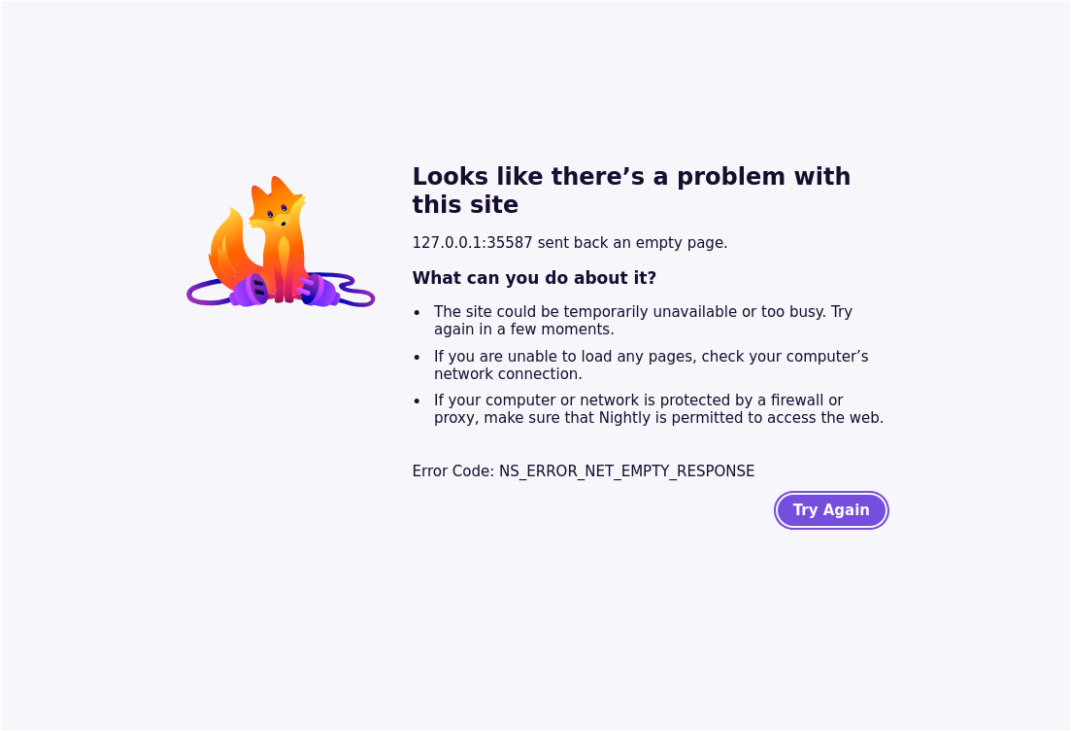
Socket-Tests bestätigen `index.html`, Unterverzeichnisse und binäre Dateiinhalte. Kodierte Traversal-Versuche und Symlinks außerhalb des document root werden abgewiesen. Dritte dürfen den document root während des Betriebs nicht verändern. Der Server ersetzt keine Betriebssystem-Sandbox.

Erklärung

Ein Präfixvergleich von Pfadzeichenketten reicht nicht. Weil auch ein scheinbar harmloser Link auf einen fremden Ordner aus dem Basisverzeichnis herausführen kann, wird zuerst das tatsächliche Ziel aufgelöst und danach geprüft, ob dieses Ziel noch innerhalb des erlaubten

Verzeichnisses liegt. Erst danach darf die Datei geöffnet werden.

Mit Headless Chromium 145.0.0.0 und Firefox Nightly wird die Seite in diesem Modus nicht richtig angezeigt. Die Browser senden aktuelle Request-Zeilen, die der strikte Zweitoken-Parser ablehnt. Chromium meldete `ERR_EMPTY_RESPONSE`; Firefox `NS_ERROR_NET_EMPTY_RESPONSE`. Die beiden Browser-Screenshots zeigen die leere Antwort. Dieses Testergebnis wurde nicht als erfolgreiche Darstellung gewertet.



Aufgabe 3.5

Den Server erweitern: jede Anfrage durch einen neuen Thread behandeln, HTTP/1.0 und persistente Verbindungen unterstützen. Zeitpunkt, Request-Typ, Client-IP und Clientport zentral in einem eigenen Thread alle fünf Sekunden ausgeben. Geeignet synchronisieren.

Abgabe: `http_server.py -mode 10`. Ein Verbindungsthread liest vollständige Anfragen. Für jede Anfrage startet er einen eigenen Worker. Vor dem nächsten Request auf derselben Verbindung wartet er auf dessen Antwort. Andere Verbindungen arbeiten parallel. `GET` liefert Dateidaten, `HEAD` nur die passenden Header.

Erklärung

Ein Thread pro Verbindung allein wäre hier nicht die wörtliche Anforderung. Deshalb gibt es zusätzlich den Request-Worker. Die Reihenfolge bleibt trotzdem fest: Antwort zwei darf auf derselben TCP-Verbindung nicht mitten in Antwort eins geschrieben werden.

Antworten verwenden `HTTP/1.0`, `Content-Length`, `Content-Type` und `Connection`. Nur explizites Keep-Alive hält die Verbindung offen; `close` hat Vorrang. Die Grenze liegt bei 100 Requests beziehungsweise fünf Sekunden Inaktivität. HTTP/1.1-Request-Zeilen werden für moderne Clients akzeptiert, aber mit HTTP/1.0 beantwortet. Request-Bodies, Chunked-Encoding und doppelte Header werden abgelehnt. Fehler erhalten 400, 403, 404 oder 405 entsprechend der Ursache.

Erklärung

Eine persistente Verbindung endet nicht nach jeder Datei. Der Client braucht deshalb die exakte `Content-Length`, um das Ende der Dateibytes und damit den möglichen Beginn der nächsten Antwort zu finden, ohne dafür auf einen Verbindungsabbruch warten zu müssen. Request-Bodies werden nicht still ignoriert: Übrig gebliebene Bodybytes könnten sonst als nächste Anfrage gelesen werden.

Die Handler schreiben ausschließlich Logereignisse in eine threadsichere Queue. Ein separater Logger-Thread sammelt sie und schreibt alle fünf Sekunden Zeitstempel, Methode, Clientadresse und Clientport. Beim regulären Beenden wird der Rest ausgegeben. Nur dieser Thread schreibt den Logstrom. Passwörter und Header werden nicht erfasst.

Erklärung

Die Queue trennt das Bearbeiten der Anfrage von der Ausgabe. Statt dass jeder Request-Thread auf ein langsames Terminal und einen gemeinsam gehaltenen Datei-Lock warten muss, legt er seinen Eintrag in der Queue ab und überlässt das spätere Sammeln und Schreiben vollständig dem Logger. Dessen Puffer verändert kein anderer Thread.

Im HTTP/1.0-Modus lieferten Chromium und Firefox den Text „It works...!“ und beide Bilder korrekt. Die Bildprüfung bestätigte geladene Bilder mit positiver natürlicher Breite. Zwei Requests auf derselben TCP-Verbindung und 16 parallele Requests wurden separat getestet. Die Browser-Aufnahmen liegen im Ordner `screenshots`; das Erklär-PDF enthält zusätzlich die Chromium-Darstellung.



It works...!

Prüfung, Quellen und Abgabe

32 Tests liefen mit Python 3.8.20 und 3.14.7 erfolgreich durch; Browsersteuerung mit Python 3.11.15. Getestet wurden Anmeldung, Dot-Stuffing, Größen, Sperren, Lösch-Commit, Rollback samt erneuter Anmeldung nach Abbruch, ungültige HTTP/0.9-Steuerzeichen, leere RETR-Antworten, TOP ohne Nachrichtenkörper, Framing, Pfadgrenzen, HTTP-Fehler, Keep-Alive, Headerfehler, Inaktivitätstimeout, Parallelität, Loggerausfälle vor und während des Serverstarts sowie das Warten auf laufende Handler beim Schließen. Startbefehle und Einzelheiten stehen in `README.md`. Die Programme und Tests sind Teil der Abgabe.

Grundlagen: Moodle-Blatt 03, bereitgestellte Testdaten sowie Vorlesungsfolien zu HTTP, Mail und Python-Sockets (Seiten 10–11, 36–37 und 59–60).

POP3 und IMAP: RFC 1939 (Abschnitte 3–7), RFC 2449 (4–6) und RFC 9051.

HTTP und TLS: RFC 1945 (Abschnitte 5–6) und RFC 8314.

Moodle nennt für beide Gruppen den 17.11.2026, 08:00 Uhr (Stand 19.10.2026). Gruppe 1 ist Raphaels Angabe; die Zuordnung zu einer der beiden Moodle-Abgabeaktivitäten und die Gruppenmitgliedschaft wurden nicht geprüft. Es wurde nichts auf Moodle eingereicht.